Electric Vehicle Infrastructure Peak Load Forecasting

Comprehensive Capstone Project Technical Documentation & Progress Report

Author: EMIL **Environment:** PySpark | PyTorch | Python 3.x **Status:** Phase 1 & 2 Complete

1. Executive Summary & Project Objectives

Rapid deployment of Electric Vehicle (EV) charging infrastructure introduces severe non-linear demand spikes on municipal power grids. This capstone project develops an end-to-end Big Data ETL and Deep Learning predictive engine designed to forecast hourly peak grid demand ($t+1$) across multi-year operational horizons. By unifying transactional charging logs with localized meteorological readings, the platform provides utility distribution operators with high-accuracy peak load estimates.

Primary Strategic Objectives:

- **Distributed Harmonization:** Merge unaligned high-frequency EV event logs with continuous weather data using PySpark.
- **Sequence Reconstruction:** Enforce strict continuous 24-hour temporal sequences with zero-imputation to enable sequential deep learning.
- **Spatio-Temporal Hybrid Modeling:** Implement a hybrid Conv1D + Stacked LSTM architecture in PyTorch to simultaneously capture short-term weather/load spikes and daily cyclic rhythms.

2. Dataset Overview & Data Processing Pipeline (ETL)

The pipeline integrates two distinct data streams spanning June 2019 through December 2022:

- **EV Charging Sessions (`evwatts.public.session.csv`):** High-frequency transactional logs capturing start times, duration, and energy consumed ($ ext{kWh}$).
- **Local Weather Metrics (`weather_data.csv`):** Hourly meteorological features (temperature, relative humidity, precipitation, direct radiation) retrieved from Open-Meteo.

2.1 PySpark Distributed ETL Architecture

To handle timestamp mismatch and unobserved charging hours without creating gaps in temporal sequence data:

1. **Hourly Downsampling:** Converted millisecond-level timestamps to hourly resolution using PySpark's `date_trunc("hour", ...)`.
2. **Distributed Aggregation:** Computed hourly grid metrics: $ ext{total_kwh_demand} = \sum ext{energy_kwh}$.
3. **Continuous Sequence Generation:** Created an unbroken 30,783-hour grid spine using Spark SQL's `sequence()` and `explode()` functions.
4. **Explicit Zero-Imputation:** Performed a `LEFT JOIN` from the grid spine onto charging data, using `coalesce(col("total_kwh_demand"), lit(0.0))` to assign $0.0 ext{ kWh}$ to unobserved hours.

3. Engineered Feature Matrix (11 Predictors)

A 11-feature matrix was constructed to provide environmental, calendar, and autoregressive signals:

Category	Feature Name	Computation / Transformation	Analytical Rationale
Environmental	`temperature_2m`	Open-Meteo API	Battery thermal charging efficiency
Environmental	`relative_humidity_2m`	Open-Meteo API	Seasonal climate variations

Category	Feature Name	Computation / Transformation	Analytical Rationale
Environmental	`precipitation`	Open-Meteo API	Severe weather travel disruptions
Environmental	`direct_radiation`	Open-Meteo API	Solar irradiance & ambient heat influence
Calendar Signal	`hour` (0–23)	PySpark `hour()`	Captures 24-hour diurnal demand cycles
Calendar Signal	`day_of_week` (1–7)	PySpark `dayofweek()`	Captures weekly commuter routines
Calendar Signal	`month` (1–12)	PySpark `month()`	Captures annual seasonal drift
Calendar Signal	`is_weekend` (0/1)	Binary Flag (day in [1,7])	Distinguishes workday vs weekend patterns
Autoregressive	`kwh_lag_1h`	PySpark `Window.lag(1)`	Captures immediate load momentum ($t-1$)
Autoregressive	`kwh_lag_24h`	PySpark `Window.lag(24)`	Captures daily seasonal recurrence ($t-24$)
Rolling Trend	`kwh_rolling_avg_24h`	PySpark `avg().over(24h)`	Smooths short-term noise to establish baseline

4. Hybrid Conv1D + LSTM Model Architecture

A spatio-temporal hybrid neural network was implemented in PyTorch, combining local convolutional feature extraction with sequential LSTM memory:

```
                Input Sequence: (Batch Size, 24 Hours, 11 Features)
                                        ↓
[ Conv1D Layer: 64 Filters, Kernel Size=3, ReLU ] ← Local Weather & Load Spike Feature Extraction
                                        ↓
[ 2-Layer Stacked LSTM: 64 Units, Dropout=0.2 ] ← Long-Term Sequential & Diurnal Memory
                                        ↓
        [ Linear Dense Output Layer: 1 Unit ] ← Next-Hour Grid Load Forecast (t+1)
```

Training Parameters:

- **Sliding Window:** $SEQ\_LEN = 24$ lookback hours.
- **Train/Test Split:** Sequential $80\%/20\%$ split ($24,626$ train hours vs $6,157$ test hours).
- **Standardization:** `StandardScaler` fitted strictly on training partition.
- **Optimization:** Adam Optimizer ($ ext{lr} = 0.001$), Mean Squared Error (MSE) loss over 15 Epochs.

5. Summary of Key Performance Indicators (KPIs)

KPI Metric Category	Metric Description	Recorded Value
Data Scope	Total Unbroken Time-Series Horizon	**30,783 Hourly Rows**
Storage Volume	Optimized Feature Matrix Size	**1.72 MB** (Single Parquet)
Feature Matrix	Total Input Predictors	**11 Features**
Training Loss	Final Convergence Loss (Epoch 15)	**0.05183 MSE**

KPI Metric Category	Metric Description	Recorded Value
Grid Load Extrema	Observed kWh Demand Range	**~2,000 kWh (Troughs) to ~22,000 kWh (Peaks)**

6. Diagnostic Insights & Model Performance

- **Diurnal Alignment:** Test evaluation across a 7-day window demonstrates that the Conv1D + LSTM model precisely tracks peak and trough transitions without phase lag.
- **Base Load Accuracy:** Residual analysis shows tight error clustering near $0.0 ext{ kWh}$ for standard operating conditions ($< 5,000 ext{ kWh}$).
- **Peak Surge Anomaly (Heteroscedasticity):** Residual scatter plots revealed expanding variance at extreme peak surges ($> 15,000 ext{ kWh}$). This finding provides strong academic justification for the upcoming model optimization phase.

7. Roadmap for Next Phases

1. **Peak Surge Optimization:** Implement $\log(1 + y)$ target scaling or asymmetric/Huber loss functions to penalize underprediction of high demand spikes.
2. **PySpark MLlib Comparative Benchmarking:** Train Linear Regression, Decision Tree, and Random Forest / GBT Regressors in Spark MLlib to evaluate baseline performance against the Deep Learning approach.
3. **API Orchestration:** Package model inference into an API serving layer (e.g., MuleSoft integration) for real-time utility grid querying.